NATIONAL ECONOMICS UNIVERSITY · BFT65B – BIG DATA ANALYTICS IN FINANCE

Final Examination – Individual Research Report

A Big-Data Pipeline for Bank Failure Early Warning

1. Context

You are a data engineer at Vietnam's central bank. The supervision department plans an **early-warning system** that ranks banks by failure risk, and it first needs a **reliable, scalable and reproducible data pipeline**. Vietnamese supervisory data are not public, so you will build the pipeline on US data. This is a **data-engineering** project: most of the grade is for the pipeline, not for model accuracy. Answer three questions:

Q1. How should four decades of bank reports be stored and processed on a laptop, and when would a Spark cluster be needed?

Q2. What quality problems do the raw data contain, and how does the pipeline handle them without look-ahead bias?

Q3. Can the pipeline's output support an honestly validated early-warning model?

2. Data

2.1 The Dataset

The US Federal Deposit Insurance Corporation (FDIC) publishes, free of charge, the quarterly financial reports (*Call Reports*) of every insured institution and the history of bank failures. The data cover more than 40 years and three crisis periods (late 1980s to early 1990s, 2008–2012, 2023). There are three tables, linked by `CERT`:

Table	Content	Example fields
Financials	One row per bank and quarter (`REPDTE`); about 1.68 million rows	`ASSET`, `DEP`, `EQ`, `NETINC`, `ROA`, `NCLNLS`, `LNATRES`, `RBC1AAJ`
Failures	Every failed bank; about 4 100 rows	`CERT`, `FAILDATE`, `COST`
Institutions	Every bank, including why it stopped reporting	`CERT`, `STNAME`, `ACTIVE`, `CHANGEC1`

Amounts are in thousands of USD and ratios in percent. Field definitions: `https://api.fdic.gov/banks/docs/`.

2.2 Obtaining the Data

Any documented, reproducible method is allowed: the public API (no key needed, `https://api.fdic.gov/banks`), the bulk-download pages (`https://banks.data.fdic.gov/bankfind-suite/bulkData`), or the optional starter script `fdic_download_starter.py` from the course page. A full API download takes about one hour. **Everyone uses the same scope:** Financials for all banks from 1984Q1 to 2026Q2, plus the complete Failures and Institutions tables. **Which fields you use is your decision**: choose them from the field dictionary and justify the choice.

2.3 Label and Test Set

Label: $y_{i,q} = \mathbf{1}\{\text{bank } i \text{ fails in } (q,\ q+4 \text{ quarters}]\}$; mergers are not failures. For modelling, use quarters up to **2025Q2** only, because later labels are incomplete. **Split (your choice):** the test period comes **after** the development period, with a gap of at least **four quarters**, and contains **at least 50** failures (failures cluster in 1984–1993 and 2008–2014). Justify your choice. Tuning, feature

selection and the threshold are decided on the development period only, and the test period is used **once**.

Tools. pandas/NumPy, scikit-learn, Polars, PyArrow and PySpark. Polars handles the full panel on a laptop. PySpark is for **demonstration** and may run locally, on Google Colab or on Databricks Free Edition. No paid cloud is needed.

3. Tasks

For each part, explain your design choices and support them with tables, figures or measurements. The total is **100 points**.

Part 1 – Acquisition & Storage [15 pts]

Acquire the data with a reproducible (ideally resumable) procedure and verify that it is complete. Design an efficient storage layout (format, schema, partitioning, compression) and show with measurements why it is better than storing raw CSV.

Part 2 – Data Quality & Panel [20 pts]

Produce an automated data-quality report, identify the problems in the raw data, and justify how you handle each one. Build a clean bank–quarter panel that treats exited banks correctly (failure vs. other exits) and is **point-in-time** correct.

Part 3 – Scalable Processing [25 pts]

Implement the core transformations efficiently with **Polars** and explain how the engine executes them. Benchmark your choices against a pandas baseline. Reproduce key steps in **PySpark**, check that the results match, and interpret the execution plan. Argue, from your measurements, when a cluster would be justified.

Part 4 – Feature Pipeline [15 pts]

Design the features you consider informative for bank risk, build a leak-free feature table joined with the label, and put all preprocessing in a pipeline fitted on training data only.

Part 5 – Modelling & Validation [15 pts]

Compare a simple baseline with your own model(s), including a short **Spark MLlib** demo. Choose a validation scheme and metrics suited to time-ordered, highly imbalanced data, and justify them. Set the alarm threshold from the costs of false alarms and missed failures, estimated from the data. Evaluate once on the test period.

Part 6 – Conclusions [5 pts]

Answer Q1–Q3, state the limitations, and explain what would change for Vietnam.

Reproducibility [5 pts]

The pipeline reruns end to end from the README.

4. Submission

1. **Report** in **English**, as a PDF or DOCX; at most **20 pages** plus a 10-page appendix. The first page gives your name, student ID, **GitHub URL**, the download method and date, row counts, and the development and test periods with their numbers of failures.
2. **GitHub repository**, **public** from the deadline until grades are released (a private or broken link counts as no repository; later commits are ignored). It contains a `README.md`, a pinned

`requirements.txt`, and the code or notebooks (with outputs and fixed seeds). **Do not commit data files.**

3. **Academic integrity.** Cite every source you use (code, papers, AI tools) in the report and the README. Uncited copying is **plagiarism**.

5. Grading Rubric

Part	Pts	Excellent (85–100%)	Adequate (50–84%)	Insufficient (<50%)
1. Storage	15	Complete, verified data; efficient layout justified by measurements.	Choices not justified.	Incomplete data or raw files only.
2. Quality	20	Problems found systematically; sound handling; exits and point-in-time treated correctly.	Partly done.	No audit.
3. Processing	25	Efficient Polars code with engine behaviour explained; fair benchmark; Spark results match; scaling argued.	Works, but shallow analysis.	No Polars or no Spark.
4. Features	15	Well-motivated, point-in-time features; preprocessing fitted on training data only.	Some gaps.	Uses future data.
5. Modelling	15	Validation and metrics suited to time-ordered, imbalanced data; data-based threshold; single test; MLlib demo.	Weakly justified choices.	Random CV, accuracy only, or test reused.
6. Conclusions	5	Evidence-based.	Generic.	Missing.
Reproducibility	5	Public and rerunnable.	Runs with effort.	Not reproducible.

Leakage gate. Any use of future information caps **Parts 4 and 5 together at 30%** of their points.